

Coding Discussion: Priority Queues and Randomly-Mergeable Binary Heaps

Design and Analysis of Algorithms

Brian C. Dean

**School of Computing
Clemson University**



Priority Queues

- In a FIFO queue, elements exit in the same order as they enter.
- In a priority queue, the element with highest priority is first to exit.
- Operations supported by all priority queues: *insert*, *remove-min*
- Extended operations: *delete*, *decrease-key*, *increase-key*
- Many uses:
 - **Scheduling:** Manage a set of tasks, where you always perform the highest-priority task next.
 - **Sorting:** Elements exit a priority queue in sorted order.
 - **Complex Algorithms:** For example, Dijkstra's shortest path algorithm.

Priority Queue Implementations

- There are many simple ways to implement a priority queue:

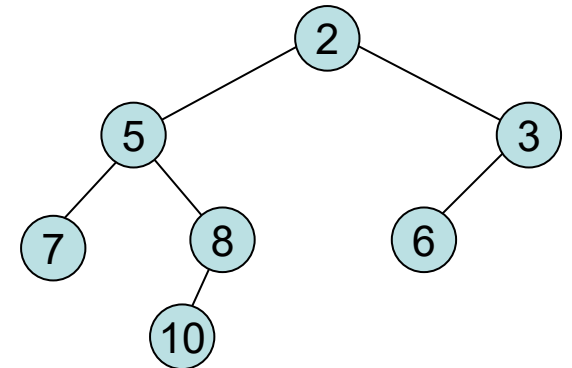
	<i>insert</i>	<i>remove-min</i>
Unsorted array or linked list	$O(1)$	$\Theta(n)$
Sorted array or linked list	$\Theta(n)$	$O(1)$
Binary heap (to discuss later)	$O(\log n)$	$O(\log n)$

- Note that in the comparison-based model, either *insert* or *remove-min* must run in $\Omega(\log n)$ worst-case time...

Heap-Ordered Binary Trees

- There are many simple ways to implement a priority queue:

	<i>insert</i>	<i>remove-min</i>
Unsorted array or linked list	$O(1)$	$\Theta(n)$
Sorted array or linked list	$\Theta(n)$	$O(1)$
Binary heap (to discuss later)	$O(\log n)$	$O(\log n)$
Randomly mergeable binary heap (today's discussion)	$O(\log n)$ in expectation	$O(\log n)$ in expectation



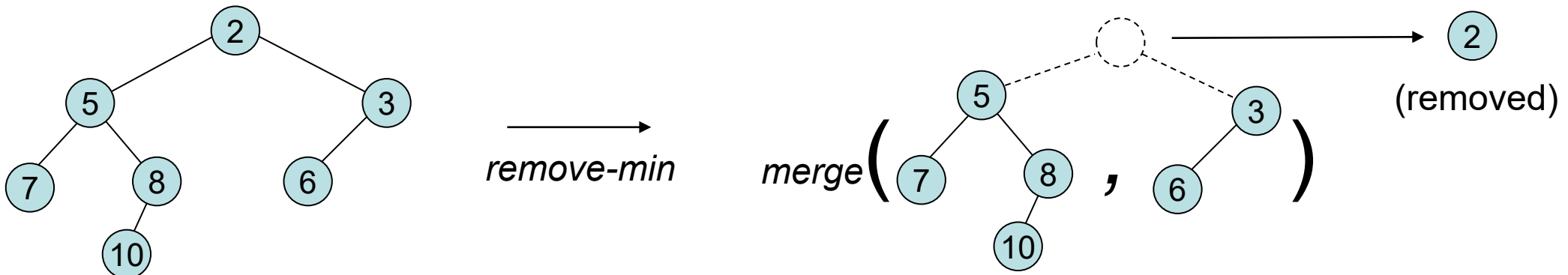
- Store elements in a “heap-ordered” binary tree.
- Each node stores key and pointers to left / right children.
- Key operation: *merge* two such trees into one.

All You Need is Merge...

(The Lesser-Known Hit Beatles Song... :)

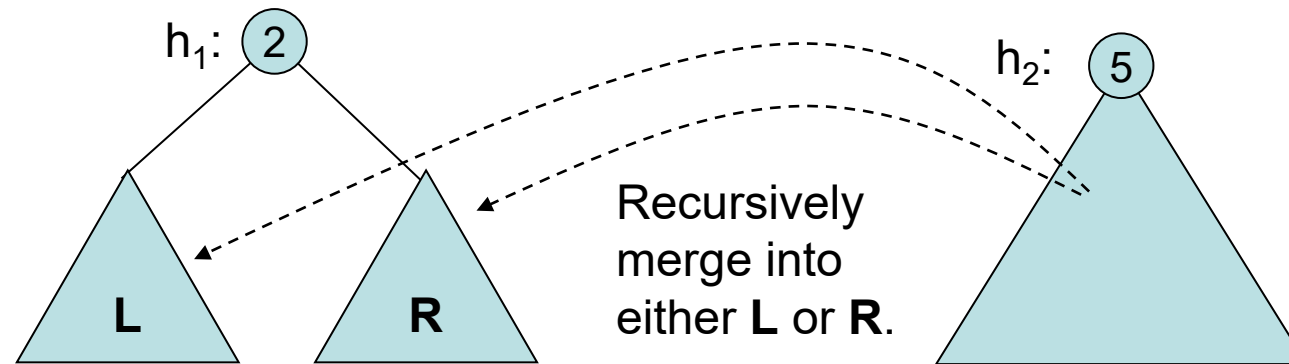
If you can *merge* two heap-ordered trees in $O(\log n)$ time, all other operations are now easy to implement in $O(\log n)$ time!

- *insert(x)*: merge with a new 1-element tree containing just x.
- *remove-min*: remove root, merge left & right subtrees.
- *delete(x)*: replace x with merge of its left & right subtrees.
- *decrease-key, increase-key*: delete and re-insert with new key.



Merging Two Heap-Ordered Trees (Recursive Viewpoint)

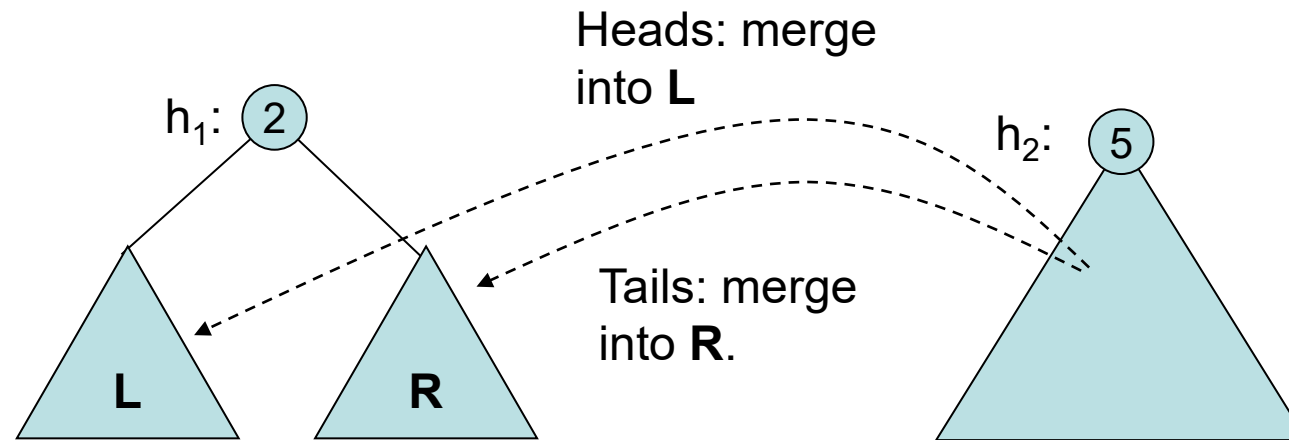
- Take two heap-ordered trees h_1 and h_2 , h_1 having the smaller root.
- Clearly, h_1 's root must become the root of the merged tree.
- Then recursively merge h_2 into either the left or right subtree of h_1 :



- As a base case, the process ends when we merge a heap h_1 with an empty heap, the result being just h_1 .

Merging Two Heap-Ordered Trees (Recursive Viewpoint)

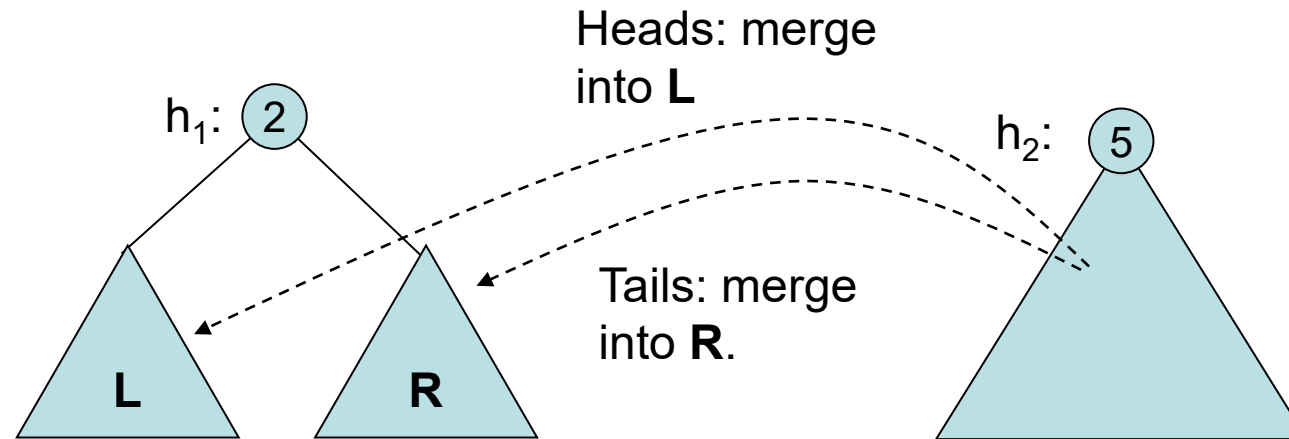
- Take two heap-ordered trees h_1 and h_2 , h_1 having the smaller root.
- Clearly, h_1 's root must become the root of the merged tree.
- Then recursively merge h_2 into either the left or right subtree of h_1 , **choosing by coin flip**:



- This makes merging (and therefore every other priority queue operation) run in $O(\log n)$ time in expectation!

An Easy Randomized Reduction Argument...

- Probability $\frac{1}{2}$ of merging with the smaller subtree in h_1 , which reduces $\text{merge}(h_1, h_2)$ to $\text{merge}(\text{something} \leq \frac{1}{2} \text{ the size of } h_1, h_2)$.



- This makes merging (and therefore every other priority queue operation) run in $O(\log n)$ time in expectation!

Merging Two Heap-Ordered Trees (Null Path Merging Viewpoint)

- Walk down a tree via coin flips, tracing some “null path”.
- Its expected length is $O(\log n)$ by the randomized reduction lemma.
- So merging along two of these paths takes $O(\log n)$ expected time.

